claude-windows / 144a31b7-7962-4fee-9d6f-8a7854daa63f

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\144a31b7-7962-4fee-9d6f-8a7854daa63f\subagents\workflows\wf_134043cd-8c3\agent-ab858a04d1bde1657.jsonl`
- SHA-256: `4884b0f626dbea20e747ff0749c7ed0ae6b605bc217629309c6e59fdb6f769e3`
- Source modified: `2026-06-20T09:14:02+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_134043cd-8c3`
- session_id: `144a31b7-7962-4fee-9d6f-8a7854daa63f`

Transcript

1. user (2026-06-20T09:02:57.842Z)

Design the KhelSutra "locally installed, UI-powered badminton rally indexer + reel generator" ? a SINGLE-BOX appliance where the web UI drives the full upload -> index -> pick rallies -> reel -> download loop ? using this lens:
REVERSE-PROXIED UNIFIED ORIGIN: a single web origin (Caddy/nginx on the box, e.g. app.khelsutra.guru) serves the static site + SPA and reverse-proxies /api/* to the engine FastAPI, with AUTH AT THE EDGE (basic/OAuth/Cf-Access) so the no-auth engine is never publicly exposed. The "appliance" is a packaged, reproducible deploy extending the SETUP_NEW_MACHINE runbook. Optimize for exposure-safety + reproducible ops + TLS/subdomain.

Owner's framing: both the website and the engine codebase now live on ONE box (already true on the droplet); augment the UI to also DRIVE the badminton-highlight-indexer so the whole thing operates as a self-contained locally-installed appliance.

GROUND EVERY DECISION in this verified research (decoupled-compute architecture, engine API/worker, hosting/exposure, UI repo). Reconcile with the decoupled architecture ? do NOT propose something that fights the StorageBackend/worker model. Be brutally honest about the CPU-only droplet (no local WASB), the no-auth API, and the shared-testbed boundaries:


```
{
  "decoupled": {
    "storage_contract": "StorageBackend ABC = backend/storage/base.py:17. Abstract methods: fetch_to_local(ref, dst=None, *, overwrite=False)->str (base.py:21), put(local_path, ref, *, content_type=None)->StorageRef (:27), exists (:32), stat (:35), list(prefix)->Iterator[StorageRef] (:38), delete (:42); plus a NON-abstract signed_url(ref,*,ttl,method) that raises NotImplementedError by default (:45-49) ? callers MUST treat that as expected and fall back to fetch_to_local (local can't sign; gdrive only emulates).\n\nURI scheme (backend/storage/ref.py:48 parse_uri + docstring :1-13): bare path or local:// => local (today's behaviour, IDENTITY passthrough, no copy); s3://<bucket>/<key> => s3 (covers AWS + Cloudflare R2 + DO Spaces + MinIO, distinguished only by endpoint_url); gcs:// (or gs://, normalized) => gcs; gdrive://<folderId>/<path> => gdrive (follow-up stub). Windows C:\\... has no :// so falls through to local. Value objects: StorageRef(backend,bucket,key,uri) frozen dataclass with a .name property (ref.py:34-45); StorageStat(size,modified,etag,content_type) (ref.py:25).\n\nThin URI helpers in backend/storage/__init__.py: get_storage(backend,config,**overrides) (:43), fetch(uri,dst,config,overwrite) (:55), put(local_path,uri,config,content_type) (:63), list_uris(prefix_uri,config) (:71). Backends are lazy-resolved (__init__.py:26 _backend_class) so `import backend.storage` never pulls boto3/google SDKs ? they import inside the concrete module (s3.py:29 `import boto3`). CREDENTIALS NEVER IN CONFIG: s3.py:8-10 + S3Storage.__init__ (s3.py:22) takes only endpoint_url/region/addressing_style/signed_url_ttl; boto3's default chain (env AWS_ACCESS_KEY_ID/SECRET, instance role, ~/.aws) supplies creds. A client can be injected for tests (s3.py:26).\n\nLocalStorage (local.py:18): fetch_to_local is identity when dst is None or dst==src (local.py:33), else shutil.copyfile; put is a copy that no-ops when src==dst (:42-47); list walks the dir (:56). So backend='local' is byte-for-byte the pre-storage-layer behaviour.\n\nBucket layout (04 doc §3, lines 56-66): videos/<md5>.mp4 (immutable, byte-
```

identical to what labels were made on), labels/<sport>/<video_id>.csv, manifest.json (the join table ? needs an md5 field added, an M2 prereq per 02 §4), weights/<semver>/{model,manifest} + weights/checkpoints/, outputs/<video_id>/{highlights.mp4, intervals.csv, report.json}.\n\nStorageConfig (backend/config/models.py:291): backend: Literal[\"local\",\"s3\",\"gcs\",\"gdrive\"]=\"local\"; output_root:str=\"\"; per-backend loose dicts local/s3/gcs/gdrive (:301-304) passed straight to the backend constructor (endpoints/regions only, no secrets).",

"worker_cli": "Entrypoint backend/worker/__main__.py ? `python -m backend.worker {infer|train}`. Docstring (:1-15): a THIN ORCHESTRATOR composing the SAME building blocks as the backend.main CLI (validator registry -> segmenter -> report), never a forked pipeline. Parser at build_parser() (:302).\n\nINFER ? cmd_infer (:88): `python -m backend.worker infer --video-uri <uri> --out-uri <prefix> [--segmenter] [--config] [--scratch DIR] [--max-frames] [--set k=v]`. Flow: (1) load_config + _apply_overrides (:49, dotted-path setattr onto the typed pydantic config, _coerce bool/int/float at :36); set config.output.output_dir=scratch (:95). (2) PULL: storage.fetch(video_uri, scratch/in.mp4) ? identity for local://, download for s3/gcs (:98). (3) IDENTITY: compute_video_id = whole-file MD5 (:101), the labels<->video join key; pulled bytes must be byte-identical. (4) VALIDATE via the SAME validator_registry as the main CLI (:105), write video row to a scratch SQLite DB (:107). (5) SEGMENT: segmenter_registry.get(seg_name or config.indexing.default_segmenter).process_video(...) (:120-127). (6) emit_indexer_report in a finally (:136, parity with main CLI, never raises). (7) SERIALIZE outputs/<video_id>/{intervals.csv, report.json} ? intervals.csv is decimal-second [start,end)+shot_count+ending_reason, the join-friendly schema matching the rally-annotator labels (_write_intervals_csv :62). (8) PUSH each file via storage.put to <out_prefix>/outputs/<video_id>/<fn> (:149-154), idempotent on video_id.\n\nTRAIN ? cmd_train (:192): `python -m backend.worker train --corpus-uri <prefix> --out-uri <prefix> --version <semver> [--trajectory-source synth|detector] [--segmenter wasb_hybrid] [--floor] [--fps] [--frame-width] [--config] [--scratch] [--set]`. Flow: list labels under <corpus>/labels/*.csv (storage.list_uris :217); REQUIRES >=2 for leave-one-video-out (:219). Two trajectory sources, train pipeline+harness identical either way (docstring :196-201): (a) synth (DEFAULT, the CPU \"mock GPU\") ? synth_trajectory_from_labels (stub_runner) makes deterministic label-consistent shuttle paths, NO GPU/WSL (:260-262); (b) detector ? REAL trajectories: resolve a DetectorRunner once (_resolve_train_detector :162), index videos/ by stem, fetch each video, probe real fps/width, runner.run_predict -> trajectory CSV (:242-258). Build manifest.json of specs (:269), then SHELL OUT (subprocess, because backend must NOT import training) to `python -m training.gen0.harness --manifest <m> --out <dir> --version <semver> [--floor <f>]` (:276-283). PUSH the versioned bundle to <out_prefix>/weights/<version>/<fn> (:289-294).\n\n_resolve_train_detector (:162) reuses the segmenter's _resolve_runner seam so worker and live CLI build the detector identically; rejects a non-trajectory segmenter (:179) and runs runner.healthcheck() before use (:184). Config knobs: --config (config.json), --set dotted overrides; the validated GPU-free/secret-free combo is RALLY_DETECTOR_IMPL=stub + --set indexing.skip_ai_handoff=true + storage.s3 settings (docstring :8-11, HANDOFF.md).",

"compute_abstraction": "Two ORTHOGONAL axes, deliberately separated (07 doc §2, lines 27-39): (A) ComputeBackend = WHERE a job runs (coarse, per-job: local | hosted | byo_worker / DO droplet / serverless) ? picks the machine; (B) DeviceContext = HOW a GPU stage runs ON that machine (fine, spec-aware, per-stage: probe the real GPU, resolve batch/precision/chunk knobs). Conflating them is the named design trap.\n\nSTATE TODAY: ComputeBackend is PLANNED ONLY ? `grep ComputeBackend backend/` returns nothing; it is named in PLATFORM_ARCHITECTURE.md (registry local|hosted|byo_worker) and inventoried as a FUTURE registry alongside StorageBackend in 07 §1 (:19) and 01 §3.4. NO ComputeBackend code, NO DeviceContext/DeviceSpec/DeviceProfile code ? 07 doc is a PROPOSAL (status :3) to be built at M3.5, NOT M0 (07 §6 :153, §5 caveat 2 :150).\n\nThe compute MAP / GPU-vs-CPU contract (07 §3 table lines 45-56, the load-bearing rule :57-59): \"a GPU-bound stage is a pure, restartable unit whose only job is frames -> artifact (the Frame,Visibility,X,Y CSV or a feature JSON); everything downstream is device-agnostic CPU that reads that artifact.\" GPU work is concentrated in the detection substrate ONLY ? person detection (torchvision, clean CPU fallback) + shuttle trajectory (WASB torch / TrackNet TF). Validators, ffmpeg chunk/stitch (libx264, nvenc optional), ByteTrack, windowing/candidate-merge, Gemini handoff (network I/O), DB/eval/calibration, Gen-0 head training (~4.5GB, minutes) are CPU/device-agnostic. This artifact boundary is EXACTLY where GPU and CPU work can live on different machines (the decoupling), and is already how EXPERIMENT_HARNESS/GOLDEN_DATA split today (07 §3 point 2).\n\nWhat abstracts compute placement TODAY (built): (1) the storage URI scheme ? local vs s3/gcs decides whether bytes are co-located or remote, transparently to the pipeline; (2) the detector_impl seam in trajectory_hybrid.py::_resolve_runner (:65): RALLY_DETECTOR_IMPL env (precedence) or indexing.detector_impl in {auto=WSL via _build_runner (:86), stub=CPU mock StubDetectorRunner (:77-81), native=Linux-native via _build_native_runner (:82-85)}. The native branch is the M3.5 hook but the base _build_native_runner REFUSES with

NotImplementedError (\"only WASB has a Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'\", trajectory_hybrid.py:60-63) ? so LOCAL GPU = WSL today (auto), REMOTE/Linux GPU box = native (not yet built), CPU/CI = stub. The proposed DeviceContext layer (DeviceSpec/DeviceKnobs/DeviceProfile registry, 07 §4-5) would replace scattered device= logic and fix WASB's hardcoded device='cuda' with no CPU fallback (the one inconsistency, 07 §1 :16) ? but it only bites once the native runner lands, hence deferred to M3.5.",

"built_vs_next": "BUILT & MERGED to master (PRs #246-#254, suite 936 green per HANDOFF.md:9): the full bucket-driven train->infer pipeline running E2E on a CPU box with the GPU MOCKED (deterministic stub), validated on a DO droplet reading/writing DO Spaces:\n- Storage layer (#249): backend/storage/ ? local + S3 (AWS/DO Spaces/R2/MinIO) + GCS, URI-driven; gdrive is a stub.\n- Worker dispatcher (#251 infer, #254 train): backend/worker/__main__.py ? pull -> SAME pipeline -> push.\n- CPU-mock detector seam (#247): backend/pipeline/detectors/stub_runner.py via RALLY_DETECTOR_IMPL=stub / indexing.detector_impl=stub.\n- Cross-platform canonicalization (#252); GPU-box provisioning scripts (#253): deploy/digitalocean/{bootstrap,mount_scratch,gpu_setup}.sh (NVMe scratch + native WASB conda env setup) ? scripts exist but the native runner that uses them does not.\n- Plan directory (#246): docs/DECOUPLED_COMPUTE/.\nVerified E2E (CPU, no GPU, no Gemini key, HANDOFF.md:26-30): worker train pulled 7 labels from Spaces -> synth_trajectory_from_labels (label-consistent CPU mock) -> shelled to training.gen0.harness -> LOVO over 7 videos (F1 1.000 on the mock, gate=no-ship by design) -> weights/ to Spaces; worker infer pulled a real video -> stub -> outputs/<md5>/ to Spaces.\n\nNEXT BUILD = M3.5, the ONE remaining piece for real (non-mock) training/inference (HANDOFF.md:50-65, 04 doc §M3.5 :98): build NativeWasbRunner(DetectorRunner) in backend/pipeline/detectors/ ? sibling to WasbRunner but WITHOUT WslCommandMixin: run WASB in-process/native (no wsl / conda activate / to_wsl_mnt_path), device cuda/mps/cpu, weights via env (not ~/models), emit the IDENTICAL Frame,Visibility,X,Y CSV. Wire it into the existing native branch by overriding _build_native_runner in the WASB segmenter (today the base refuses, trajectory_hybrid.py:57-63). Acceptance gate = byte-parity: same clip -> WSL CSV (Windows) == native CSV (Linux) + identical candidate windows. Then wire a real-extract path into worker train/infer (--trajectory-source detector already exists; only the trajectory source flips, harness+push unchanged). Also proposed-not-built at M3.5: the DeviceContext/DeviceSpec/DeviceProfile device layer (07 doc) and the ComputeBackend registry. STILL NOTHING: no Dockerfile anywhere (README §96 verification), no cost ledger / cost columns on the jobs table (05 doc), no remote inference endpoint / API-key-gated serverless, no bucket-triggered retrain flywheel. Gen-2 VLM fine-tune is explicitly OUT OF SCOPE for the $200 budget.",

"single_box_fit": "There is NO literal \"single-box profile\" named in these docs ? the architecture is a control-plane / data-plane / compute-worker split (04 doc §1 :12-27), and a SINGLE BOX is simply the DEGENERATE configuration of that same split where all three planes collapse onto one machine. The decoupling docs frame the END STATE (separate rented GPU + remote bucket); the single-box case is what already runs.\n\nHOW it maps, concretely:\n- STORAGE = LOCAL. Set storage.backend='local' (the DEFAULT, config/models.py:298). Then storage.fetch is an identity passthrough (no copy ? local.py:33) and storage.put no-ops when src==dst (local.py:42-47); parse_uri treats bare/Windows paths as local (ref.py:48-57). The worker's pull->pipeline->push collapses to \"read local file -> run -> write local dir\", byte-for-byte the pre-storage behaviour. FULLY BUILT, and the default.\n- WORKER IN-PROCESS / CO-LOCATED. `python -m backend.worker {infer|train}` runs the SAME registries as backend.main in the SAME process (worker docstring :12-15); the control plane (the shipped async jobs table + ThreadPoolExecutor on the owner's box, PLATFORM_ARCHITECTURE.md:60,475 ? single-tenant, local-only) and the compute worker are the same machine. The owner's CURRENT Windows/WSL dev box IS a single box: control plane + storage(local) + worker + GPU(WSL) all co-located. SUPPORTED today.\n- COMPUTE on a single box. The detector_impl seam picks the local detector: auto=WSL (the owner's GPU today) or stub=CPU. No ComputeBackend needed because there is only one machine ? ComputeBackend (WHERE) is the thing you ADD when the GPU moves off-box; on a single box it is identically the no-op \"local\" choice that need not exist yet (07 §2, 01 §3.4).\n- The DO CPU droplet 168.144.126.176 is ALREADY a working single-box instance of this in the cloud: BOTH /srv/khelsutra-site AND ~/rally (engine) live on that ONE box, and it ran the full train+infer E2E (GPU mocked + storage pointed at Spaces). Swap storage.s3->local and that same box is a pure single-box deployment.\n\nWHAT a single box ALREADY supports: the entire infer + Gen-0 train flow, end-to-end, mocked-GPU, on CPU; and on the owner's Windows+WSL box the REAL WSL detector via detector_impl=auto. WHAT IS MISSING for a single box to do REAL training/inference WITHOUT WSL (i.e. a single Linux GPU box): the M3.5 NativeWasbRunner ? until it lands, a non-Windows single box can only run stub (CPU mock) or must shell to WSL (Windows-only). Everything else a single box needs (storage=local default, worker, validators, Gen-0 harness, ffmpeg) is built. The cost-ledger / API-key / serverless / Dockerfile gaps are about MULTI-tenant remote serving, not the single-box path ? a single box does not need them.",

```
    "file_refs": [
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\README.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\HANDOFF.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\01-EXISTING-PLANS-AND-GAP.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\02-COMPUTE-MAP-AND-CONTRACT.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\03-PLATFORM-OPTIONS.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\04-BUILD-AND-TEST-PLAN.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\07-COMPUTE-ABSTRACTION.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\worker\\__main__.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\__init__.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\base.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\ref.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\s3.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\local.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\config\\models.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\segmenters\\trajectory_hybrid.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\stub_runner.py"
    ]
  },
  "engineApi": {
    "process_flow": "FULL LOOP today (single FastAPI box, started via `python -m backend.main
--server` -> main.py:632-638 uvicorn on 127.0.0.1:8000). Step by step:\n\n1. UPLOAD (optional,
browser-side; chunked island in backend/api/uploads.py). A browser file is uploaded in
sequential parts <= security.max_part_mb (~95MB, edge-proxy cap): POST /api/upload/init
(uploads.py:64) validates extension (mp4/mov) + size, opens a `.partial-<id>` temp under
security.upload_dir (output/uploads) and returns upload_id + next_part URL; PUT
/api/upload/{id}/part/{i} (uploads.py:89) appends raw bytes, parts STRICTLY sequential (409 on
gap), enforces per-part + whole-file caps (413 aborts); POST /api/upload/{id}/complete
(uploads.py:123) verifies part count, os.replace into upload_dir/<filename> (de-dupes name
collisions), computes video_id = MD5 (compute_video_id) and returns {path (absolute), video_id,
size_bytes, next:\"POST /api/process with this path\"}. DELETE /api/upload/{id} aborts + deletes
partial. Upload state is IN-PROCESS only (uploads.py:_uploads dict + lock) -> a restart drops
in-flight uploads (by design, single-box alpha). NOTE: the local /static UI does NOT use these
endpoints; it just takes a server-local path string.\n\n2. PROCESS submit (main.py:240 POST
/api/process, status_code=202). Fast-fail checks run synchronously in the request:
validate_input_path (+ optional security.allowed_video_root jail), os.path.exists -> 404,
unknown segmenter -> 400. Then it persists a job row in state 'queued' (db.create_job,
main.py:264) and fires `_get_executor().submit(_drain_due_jobs)`; returns 202 {job_id,
status:\"queued\", poll:\"/api/jobs/{id}\"}. ALL slow work ? including MD5 hashing of the file ?
happens on the worker thread (#16), not in the request.\n\n3. INDEX (worker thread,
_execute_processing main.py:125). Stages surfaced via progress callback (hashing -> validating
-> segmenting(<name>) -> finalizing): compute_video_id; db.get_video to detect a re-ingest;
registry.run_all_with_context runs the pluggable validators (resolution/fps/blur/scene-
cuts/court-relevance from config.validation). On any validation failure -> upsert video status
'failed', return failed result (no destruction of prior good segments). On pass -> upsert
'processed', resolve segmenter class from segmenter_registry, record pre-run watermarks
(db.segment_watermarks), run segmenter.process_video(video_id, path, player_pool, max_frames).
Destroy-AFTER-confirm: on non-empty result prune the prior rows, on empty/raise restore prior
rows (_finalize_reingest main.py:107). A per-video observability report is always emitted in
finally: (emit_indexer_report) and grafted as result[\"reports\"]. Worker records terminal state
via db.mark_job_done (result carries the pipeline verdict in result[\"status\"]) or
db.mark_job_failed.\n\n4. POLL (main.py:274 GET /api/jobs/{job_id}). Returns {job_id, status
(queued|running|waiting|done|failed = EXECUTION state), progress, video_id, error, result,
reports, created/started/finished_at}. UI loops every 3s until status done|failed; the
pipeline's own success/failed-validation verdict lives in result[\"status\"], distinct from job
status.\n\n5. QUERY (main.py:331 POST /api/query) -> db.query_play_segments(video_id,
```

{server,receiver,duration_min,event_type}) -> {play_segments:[...]} (each with id, start_time, end_time, server, receiver, events, metadata.shot_count).\n\n6. COMPILE reel (main.py:342 POST /api/compile {video_id, segment_ids[], output_filename}). Looks up the video, intersects requested segment_ids with stored segments, applies stitching.min_shot_count floor (unknown shot_count exempt), builds (start,end) pairs, runs VideoCompiler(output_dir, begin/end_padding, watermark) -> writes reel into output_dir; returns {status:\"success\", filepath}. safe_output_filename rejects path traversal.\n\n7. DOWNLOAD reel (main.py:307 GET /api/highlights/{video_id}/{filename}) -> streams the compiled file from output_dir via FileResponse (video/mp4 or video/quicktime). The /static UI builds this URL after a successful compile to offer a download link.\n\n(There is ALSO a fully-decoupled path the FastAPI does NOT use: `python -m backend.worker {infer|train}` pulls inputs from the StorageBackend (backend/storage/), runs the SAME pipeline, pushes results back ? that is the engine-decoupling track, separate from this FastAPI loop.)",
    "endpoints": [
      "GET / (main.py:52) -> serves backend/api/static/index.html (the SPA shell)",
      "GET /api/config (main.py:121) -> returns the live typed AppConfig (global_config) as JSON",
      "POST /api/process (main.py:240, 202) -> req {video_path, player_pool?, max_frames?, segmenter?}; fast-fail path/exists/segmenter checks, creates 'queued' job, kicks a drain; resp {job_id, status:'queued', poll}",
      "GET /api/jobs/{job_id} (main.py:274) -> resp {job_id, status(queued|running|waiting|done|failed), progress, video_id, error, result, reports, created/started/finished_at}; 404 unknown",
      "POST /api/query (main.py:331) -> req {video_id, server?, receiver?, duration_min?, event_type?}; resp {play_segments:[...]}",
      "POST /api/compile (main.py:342) -> req {video_id, segment_ids[], output_filename}; runs VideoCompiler with stitching paddings+watermark; resp {status:'success', filepath}; 404 no video / 400 no matching segs or all below min_shot_count",
      "GET /api/highlights/{video_id}/{filename} (main.py:307) -> streams compiled reel from output_dir (FileResponse); 404 if not compiled yet",
      "POST /api/upload/init (uploads.py:64) -> req {filename, total_size_bytes?}; validates ext/size; resp {upload_id, max_part_mb, next_part}",
      "PUT /api/upload/{upload_id}/part/{index} (uploads.py:89) -> raw bytes body, strictly sequential parts; resp {received_parts, received_bytes}; 409 out-of-order, 413 over-cap (aborts)",
      "POST /api/upload/{upload_id}/complete (uploads.py:123) -> req {total_parts}; assembles file; resp {path(absolute), video_id, size_bytes, next}",
      "DELETE /api/upload/{upload_id} (uploads.py:147) -> aborts in-flight upload + deletes partial; resp {status:'aborted'}"
    ],
    "job_worker": "In-process async worker, backend/api/job_worker.py (re-exported on backend.main so monkeypatch seams work). ONE module-global ThreadPoolExecutor with max_workers=1 ON PURPOSE (job_worker.py:30-38): a single self-hosted box serializes GPU work and the Gemini provider is rate-fragile under parallel uploads ? so jobs run STRICTLY ONE AT A TIME (segmenters parallelize their own AI handoff internally via ai_max_workers). NO central scheduler: the DB jobs table IS the clock. Submission (/api/process and server startup) just calls _get_executor().submit(_drain_due_jobs). _drain_due_jobs (job_worker.py:100) is one tick: loop db.claim_due_job() (atomic compare-and-set on (id,status) flipping queued|due-waiting -> running, earliest-due first, so concurrent workers never double-claim) -> _run_claimed_job for each until nothing runnable. _run_claimed_job (job_worker.py:72) reconstructs the ProcessRequest from the row, runs _execute_processing with a progress cb that writes db.set_job_progress, then db.mark_job_done(result) (terminal 'done' = worker finished; pipeline verdict is in result['status']) or db.mark_job_failed on a raise. WAIT_AND_RESUME hook: if the pipeline raises JobWaiting(delay_sec) (job_worker.py:41) the job is parked via db.set_job_waiting (status 'waiting' + next_poll_at + progress) and re-claimed when due ? but NO production segmenter raises this today, so it is dormant. Self-scheduling: after each drain, _schedule_next_drain_if_waiting (job_worker.py:124) computes db.seconds_until_next_due and _arm_wake_timer arms ONE daemon threading.Timer (clamped to _MAX_DRAIN_WAKE_SEC=60s) to re-submit a drain, so parked work resumes with no further client request. CRASH RECOVERY (main.py:556 + db.fail_stale_jobs job_worker n/a -> database.py:613): at startup any job left in 'queued' OR 'running' from a dead process is swept to 'failed' (the executor was in-process, so it's truly dead) ? pollers see a terminal state instead of a hang. CAVEAT: fail_stale_jobs sweeps only queued/running, NOT 'waiting'; startup also fires one _drain_due_jobs to reclaim due waiting jobs + re-arm the wake timer (main.py:637), but a 'waiting' job whose timer died and is not yet due relies on that startup drain + the next submit to get re-armed. Single-process only:

all this state (executor, timer, uploads dict) is per-process ? there is no multi-box coordination beyond the DB row-claim.",
    "segmenter_selection": "SELECTION: per-request `segmenter` field (POST /api/process) wins; else config default. config.json sets indexing.default_segmenter='gemini' (config.json:26) [NOTE: the Pydantic built-in default in backend/config/models.py:122 is 'yolo_hybrid', but config.json overrides it to 'gemini']. main.py:150 reads getattr(global_config.indexing,'default_segmenter','motion'). Registered names (backend/pipeline/segmenters/__init__.py): motion, gemini, yolo_hybrid, tracknet_hybrid, wasb_hybrid, fusion_hybrid (each self-registers via segmenter_registry.register). Unknown name -> 400 at submit (main.py:256) or defensive 'failed' in-worker (main.py:172).\n\nWHAT EACH NEEDS:\n- gemini (gemini.py): pure cloud. Requires GEMINI_API_KEY env (gemini.py:86-95; resolved as <PROVIDER>_API_KEY). No GPU. Uploads re-encoded chunks to Google. This is the config.json default.\n- motion: pure-CPU OpenCV motion heuristic; no GPU, no key.\n- yolo_hybrid / tracknet_hybrid / wasb_hybrid / fusion_hybrid: two-stage trajectory hybrids (TrajectoryHybridSegmenter, trajectory_hybrid.py). Stage 2 (shuttle/person trajectory -> candidate windows) needs a DETECTOR RUNNER; Stage 3 is the AI handoff (Gemini) for semantic boundaries, which needs GEMINI_API_KEY UNLESS indexing.skip_ai_handoff=true (config models.py:196), in which case candidate windows ARE the rallies ? no provider, no key, nothing uploaded (the pure `gemini` segmenter ignores skip_ai_handoff).\n\nDETECTOR RUNNER selection (trajectory hybrids only; trajectory_hybrid.py:65 _resolve_runner). Precedence: RALLY_DETECTOR_IMPL env var > indexing.detector_impl (config default 'auto'):\n - 'auto' (default) -> the real WSL runner via subclass _build_runner: wasb_hybrid -> WasbRunner (WSL2 + conda env 'wasb' + pretrained WASB weights, GPU); tracknet -> TrackNet via WSL. NEEDS a GPU + WSL2 + conda env (config indexing.wasb/tracknet blocks). This is the production path NOT available on the CPU droplet.\n - 'native' / 'native_wasb' -> NativeWasbRunner (native_wasb_runner.py) via _build_native_runner ? Linux-native, no WSL/conda-activate, GPU box (M3.5, the NEXT build). ONLY wasb/fusion(substrate=wasb) support native (trajectory_hybrid.py:60 base refuses with a clear error; tracknet/yolo native -> NotImplementedError).\n - 'stub' -> StubDetectorRunner (stub_runner.py): deterministic CPU MOCK, no GPU/WSL ? makes the whole hybrid pipeline runnable/regression-testable on a CPU box/CI (\"mock a GPU with a CPU\"). This is what the decoupled CPU-droplet e2e used (RALLY_DETECTOR_IMPL=stub + skip_ai_handoff to also drop Gemini).\nSo the only GPU-free, key-free way to exercise a hybrid end-to-end is detector_impl=stub + skip_ai_handoff=true. gemini needs the key (no GPU). motion is the only fully-local, no-key, no-GPU REAL segmenter.",
    "ui_drive_gaps": [
      "EXISTS: a static SPA (backend/api/static/index.html + app.js + style.css) already drives the CORE loop ? process by server-local path (POST /api/process + 3s poll of /api/jobs/{id}), render validation results, render rally cards, filter via /api/query, select segments, POST /api/compile, and build the /api/highlights download link. So index->reel is covered for an already-on-server file.",
      "GAP ? browser upload not wired: the chunked upload endpoints (/api/upload/init|part|complete|abort) EXIST and work, but app.js never calls them; the UI only takes a typed server-local path string. A real UI needs a file-picker that chunks <=max_part_mb, drives init/part/complete, then feeds the returned absolute path into /api/process. (init exposes max_part_mb to size the chunks.)",
      "GAP ? no library/list endpoints: there is NO GET /api/videos and NO GET /api/jobs (list). A UI can only re-find a video if it kept the video_id in memory; on reload everything is lost. Need list-all-videos (with status) and list-all-jobs (history) endpoints + a backing db query ? currently only get_job(id)/get_video(id) single-row lookups exist.",
      "GAP ? segmenter/config not selectable in UI: /api/process accepts a `segmenter` field and GET /api/config exposes the registry/default, but app.js hardcodes only {video_path, player_pool} and never lets the user pick gemini vs wasb_hybrid vs motion, toggle skip_ai_handoff, or see which detector_impl/GPU/key the box supports. A full UI needs to surface available segmenters (from /api/config or list_available) + their requirements.",
      "GAP ? query response field mismatch / no 'list all segments' refresh: app.js reads data.rallies on /api/query (line 344) but the API returns {play_segments}; and the initial card list comes only from the process result, so there is no way to re-load a previously indexed video's segments without re-processing (no GET segments-by-video endpoint).",
      "GAP ? progress is coarse + poll-only: /api/jobs returns a single free-text `progress` stage string (hashing/validating/segmenting/finalizing); there is no percentage, no per-stage ETA, no streaming (SSE/WebSocket). A real UI shows just a spinner with the stage text. Also no cancel-job endpoint.",
      "GAP ? no auth / multi-user / per-user scoping: FastAPI has NO auth and CORS allow_origins='*' (main.py:42-46); upload_dir + output_dir are global (per-user partitioning is explicitly deferred to PR-3). A hosted UI can't isolate tenants today.",

```
      "GAP ? reels not discoverable: /api/highlights/{video_id}/{filename} streams a reel only
if you already know the exact output_filename you compiled with; there is no list-compiled-
reels-for-video endpoint, so a UI can't show a gallery of previously made reels after reload.",
      "GAP ? single-box concurrency ceiling: worker is max_workers=1 in-process, so the UI must
communicate 'one job at a time' (a 2nd submit queues behind the 1st); no progress/queue-position
is exposed, and a server restart fails in-flight jobs (UI should surface 'failed: interrupted by
server restart').",
      "GAP ? decoupled storage/worker path is invisible to the UI: the StorageBackend + `python
-m backend.worker` path (S3/GCS/DO Spaces) is entirely separate from the FastAPI; the UI drives
only the local-filesystem FastAPI loop. Driving the decoupled GPU-worker loop from a UI would
need new orchestration/endpoints that don't exist yet."
    ],
    "file_refs": [
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\main.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\api\\uploads.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\api\\job_worker.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\config.json",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\config\\models.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\segmenters\\base.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\segmenters\\__init__.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\segmenters\\trajectory_hybrid.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\segmenters\\wasb_hybrid.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\segmenters\\gemini.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\infrastructure\\database.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\api\\static\\app.js",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\api\\static\\index.html",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\stub_runner.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\native_wasb_runner.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\"
    ]
  },
  "hosting": {
    "exposure_auth": "INTENDED MODEL = single-tenant, private/invite-only, NO real auth shipped
yet. The engine FastAPI has ZERO auth and a wide-open CORS: backend/main.py:42-43
`allow_origins=[\"*\"]`, `allow_credentials=False` (only one \"auth\" hit in main.py, in a
comment at :39 ? no Depends/HTTPBearer/api_key guard exists). The dev SPA proxy also runs with
no auth (khelsutra web/README.md:2 \"no auth locally (DEV_USER_EMAIL)\").\n\nThe DESIGNED (not-
yet-built) exposure model is in docs/HOSTING_PLAN.md (status \"PROPOSAL ? awaiting owner
ratification\"): Alpha = Cloudflare edge (free) ? DNS/TLS + **Cloudflare Access email-OTP
allowlist** in front of BOTH app.khelsutra.guru (Pages SPA) and api.khelsutra.guru
(HOSTING_PLAN.md:26-31); a **Cloudflare Tunnel** (`cloudflared`, outbound-only, \"No inbound
ports opened ? Home IP never exposed\" :36) reaches FastAPI :8000 on the box. Auth = backend
trusts the `Cf-Access-Authenticated-User-Email` header / verify the Access JWT for per-user
scoping (HOSTING_PLAN.md:37-38). 100 MB edge body cap ? ?90 MB chunked upload (:39). Go-live
checklist (:102-108) DEMANDS the auth/exposure hardening that does NOT yet exist: Access
allowlist ON, CORS narrowed to https://app.khelsutra.guru, rate limit + per-user upload quota,
`security.allowed_video_root`, rotate the Gemini key. Beta graduates auth to Firebase Auth + GCS
direct upload (HOSTING_PLAN.md:46-67). PLATFORM_ARCHITECTURE.md:62 confirms \"API ? no auth\" is
a known gap; auth is P1 (build-vs-buy Auth0/Clerk/Supabase, §8.3). Khelsutra marketing site is
\"private/invite-only\" (KHELSUTRA_PRODUCT_OVERVIEW.md:41, :57). NET: a publicly-reachable box
today is unauthenticated; the only intended gate is Cloudflare Access (edge), not anything in
the app.",
    "locality_model": "docs/VIDEO_LOCALITY_MODEL.md (status \"THINKING DOC\", no greenlight).
Central thesis (§1, :30): \"timestamps are the product; the video is dead weight\" ? the
customer receives {rally intervals, report, reel} (kilobytes); only the highlight compiler needs
the original full-res bytes, and that is pure ffmpeg that can run wherever the original lives.
```

TODAY (L3, §0 :10-18): the chunked-upload flow assembles the full file under `security.upload_dir` (default output/uploads), registers by MD5, and **nothing ever deletes it** (retention = acknowledged gap, OQ1); the Gemini path also ships ?1280px re-encoded chunks to Google's Files API. Current cap = `security.max_upload_mb` default **4096 MB** (§0 :21), so a 10 GB upload is impossible today.\n\nThe \"locality ladder\" (§2 table :53-60): L3 full upload (built, capped, wrong for big files) ? **L2 proxy upload** (timeline-identical ?1080p re-encode, ~1-2.5 GB, the recommended alpha default §7 :140) ? L1 chunks-only / L1-direct (chunks straight to Gemini) ? **L0 fully local** (nothing leaves; the owned on-device model ? the desktop-app north star) ? L4 sneakernet (a hard drive; how the n=3 pilot runs today). Privacy framing: \"strictly no bytes leave only at L0\" and even L0 leaks ?1280px chunks to Google while Gemini is the brain ? must be in consent language (OQ3 :129). Storing the proxy not the original = ~10× less disk + smaller privacy surface (§3 :71-72), and the review UI prefers streaming a 1080p proxy (OQ6 :132). For a UI streaming source: L2 streams the proxy (good); L1 has only chunks ? review degrades ? this argues L2 as the default playback/clip source. Reel delivery undecided (OQ7 :133): server-compiled proxy reel (shareable 1080p) vs intervals-back + client cuts full-res. Consent/marketing (KHELSUTRA_PRODUCT_OVERVIEW.md:101-103): \"handled securely on our infrastructure\", never published, minors' footage never used to improve the tool, training opt-in only.",

"droplet_layout": "ONE shared CPU droplet 168.144.126.176 (DigitalOcean, Ubuntu 24.04, 2 vCPU / 3.8 GB, **NO GPU** ? SETUP_NEW_MACHINE.md:7) hosts BOTH:\n1) **The rally engine** at `~/rally` (git clone or scp'd archive; SETUP_NEW_MACHINE.md §3 :67-77). FastAPI on **port 8000** (HOSTING_PLAN.md:31; khelsutra web/README.md:2 dev proxy ? http://localhost:8000). Bootstrap = deploy/digitalocean/bootstrap.sh (mirrors CI: ffmpeg + libgl1 + .venv + CPU-or-CUDA torch + `pip install -e .[dev]`; verified torch 2.12.1+cpu, 922 passed/11 skipped). Storage layer = backend/storage/ (base/local/s3/gcs/gdrive/ref); worker = `python -m backend.worker {infer|train}` (backend/worker/__main__.py), URI-driven. Verified e2e 2026-06-20 reading/writing real DO Spaces (sgp1), GCS emulator, MinIO ? with the GPU MOCKED (`RALLY_DETECTOR_IMPL=stub`, `indexing.detector_impl=stub`, `skip_ai_handoff=true`; SETUP_NEW_MACHINE.md §6a :120-143).\n2) **The khelsutra marketing/request-access site** at `/srv/khelsutra-site` (isolated dir; PORTABILITY.md:32), a dependency-free Node `site/server.js` (node:http + node:sqlite) on **port 8787**, under systemd unit `khelsutra-site`, live at http://168.144.126.176:8787 (khelsutra/NEXT_STEPS.md:11-12; site/scripts/droplet-smoke.sh defaults REMOTE_DIR=/srv/khelsutra-site PORT=8787). Same code also deploys to Cloudflare Workers+D1 (PORTABILITY.md two-hosts table); the VM is the \"if Cloudflare acts out\" fallback.\n\nGPU box = a SEPARATE, NOT-YET-BUILT machine (cattle): a DO **GPU Droplet**, RTX 4000 Ada 20 GB ~$0.76/hr, in a GPU region (tor1/nyc2/atl1, NOT blr1 ? you cannot add a GPU to a CPU droplet; SETUP_NEW_MACHINE.md:43-47). Port procedure §9 :226-256: provision ? code to ~/rally ? `mount_scratch.sh` (formats+mounts the 2nd raw NVMe at /scratch, ephemeral, NOT in fstab; `export TMPDIR=/scratch`) ? `bootstrap.sh --gpu` (CUDA 12.4 torch) ? `gpu_setup.sh` (a SEPARATE miniconda env \"wasb\": py3.8 + torch 1.11.0+cu113, clones nttcom/WASB-SBDT, needs wasb_badminton_best.pth.tar) ? the \"M3.5\" native-WASB enabler. The real Linux-native DetectorRunner (NativeWasbRunner) is the NEXT build, not yet landed.\n\nSECRETS (the only non-reproducible items, never in repo): GEMINI_API_KEY at /root/.keys/GEMINI_API_KEY (chmod 600) and bucket creds in ~/.aws/credentials [default] (chmod 600) (SETUP_NEW_MACHINE.md:238-244). Bucket layout: `videos/ labels/ manifest.json weights/ outputs/<video_id>/` (:210). Teardown = `doctl compute droplet delete` (cattle).",

"desktop_vision": "YES ? a \"locally installed\" cross-platform desktop app IS already envisioned, in docs/DESKTOP_APP_PLAN.md (status \"DRAFT ? owner-gated; nothing built; the entire build is [design]\", :3-7). Pitch (§0 :11-12): \"Plug in your card, get your highlights\" ? package the mature local pipeline (WASB shuttle detector + windowing + ffmpeg stitch, already runnable offline via tools/generate_highlights.py --skip-ai-handoff) for non-technical recreational players; plug in USB/SD ? one click ? get back only the reels, 100% on-device, NO upload, NO copy of the 4K originals. It explicitly = the `local`/`LocalDesktop` ComputeBackend of PLATFORM_ARCHITECTURE.md §3.2 made into a product, and realizes the **L0 tier** of VIDEO_LOCALITY_MODEL (§4.3 :101-102, and DESKTOP_APP_PLAN header :6).\n\nSTACK (§3.4 :61-64, §4.4 :104-105): app shell = lean \"CLI + installer MVP wrapping the existing `indexer` console entry / generate_highlights.py ? graduate to a **Tauri** GUI (Rust + system webview, ~3-10 MB, MIT/Apache), with the **Python/FastAPI backend bundled as a localhost sidecar via PyInstaller** (PEP 621 pyproject + `indexer` entry point already shipped, #167). Electron is the fallback. Detector = a NEW **NativeWasbRunner** (DetectorRunner subclass, NO WSL ? drops the WSL2 dependency, D3 :37) lifting backend/pipeline/detectors/wasb_infer.py to native torch/onnxruntime per-OS, keeping the identical Frame,Visibility,X,Y CSV; windowing + VideoCompiler stitch reused verbatim. Licensing (D4 :38): MIT/Apache/BSD only, ffmpeg as an **LGPL build** (openh264/hardware encoders, no GPL libx264). \n\nCRUX/gate (§0 :14, §5, P0): compute on a typical no-GPU enthusiast laptop is the make-or-break unknown ? WASB is ~3.4× real-time on the

```
owner's RTX 2070S [measured] but CPU/Apple-MPS throughput is UNVERIFIED; P0 is a feasibility
spike that must measure it before any app code, and everything past P0 is gated. Relation to
browser appliance: it's the privacy-max end of the SAME spectrum as the hosted SPA ? same
pipeline, control/data-plane \"degenerates to one machine\" (§4.3 :102); Gemini stays an
optional opt-in toggle, never required.",
    "single_tenant_assumptions": [
      "NO app-layer auth exists: backend/main.py FastAPI ships open CORS allow_origins=['*'] and
zero auth dependency ? a publicly-reachable box is wide open unless something external (the
planned Cloudflare Access edge allowlist) gates it; HOSTING_PLAN go-live checklist (:102-108)
lists CORS-narrowing + Access + rate-limit + per-user quota as REQUIRED-but-not-yet-done before
exposure.",
      "The intended gate is edge-only and email-allowlist, not real multi-tenant identity:
Cloudflare Access email-OTP for ?50 alpha testers, backend trusting Cf-Access-Authenticated-
User-Email (HOSTING_PLAN.md:37-38). Multi-tenant identity/RLS/per-tenant isolation is P1 and
unbuilt (PLATFORM_ARCHITECTURE.md §4.2, §6).",
      "Cloudflare Tunnel keeps the home/box IP unexposed and opens NO inbound ports (cloudflared
dials out; HOSTING_PLAN.md:36) ? the single box is reachable only via the edge, never directly,
in the intended design (the raw 168.144.126.176:8000/:8787 exposure today is a testbed, not the
production posture).",
      "Retention is an UNSOLVED guardrail gap: uploaded originals under output/uploads are never
deleted (VIDEO_LOCALITY_MODEL §0 :10-18, OQ1) ? a real privacy/liability hole for a single
publicly-reachable box holding many users' video; default to keeping only proxies + intervals.",
      "Minors are excluded by default from the training pool, and per-user/contributed-footage
training requires a SEPARATE explicit opt-in (CLAUDE.md guardrails; PLATFORM_ARCHITECTURE.md §7
:516-526; KHELSUTRA_PRODUCT_OVERVIEW.md:98-103) ? never train on what merely passes through a
tenant's bucket.",
      "Paid LLMs (Gemini/OpenAI/Anthropic) are inference/serving/fallback ONLY, never a training
source; even 'local' modes leak ?1280px chunks to Google's Files API on the Gemini path, which
MUST be stated in consent language (VIDEO_LOCALITY_MODEL OQ3; PLATFORM_ARCHITECTURE §5A teacher
constraint).",
      "Per-video cost ceiling needed so one box can't be surprise-billed: cap GPU/Gemini spend
per video, ai_max_workers=1, serial uploads, bounded retry (VIDEO_LOCALITY_MODEL §4 :84-103).",
      "Secrets live only on the box (GEMINI_API_KEY in /root/.keys chmod 600, bucket creds in
~/.aws/credentials chmod 600) ? never in repo/PR/config.json; rotate the previously-chat-shared
Gemini key (SETUP_NEW_MACHINE.md:238-244; HOSTING_PLAN.md:106).",
      "The droplet is a SHARED CPU-only testbed (MinIO 9000/9001, GCS emulator 9023, no GPU);
real GPU WASB is a separate cattle GPU droplet in a non-blr1 region ? do not assume GPU or treat
shared services as production."
    ],
    "file_refs": [
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\HOSTING_PLAN.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\PLATFORM_ARCHITECTURE.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\VIDEO_LOCALITY_MODEL.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\DESKTOP_APP_PLAN.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\SETUP_NEW_MACHINE.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\KHELSUTRA_PRODUCT_OVERVIEW.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\deploy\\digitalocean\\README.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\deploy\\digitalocean\\bootstrap.sh",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\deploy\\digitalocean\\gpu_setup.sh",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\deploy\\digitalocean\\mount_scratch.sh",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\main.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\worker\\__main__.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\base.py",
      "C:\\Users\\avidu\\Projects\\khelsutra\\NEXT_STEPS.md",
      "C:\\Users\\avidu\\Projects\\khelsutra\\docs\\PORTABILITY.md",
      "C:\\Users\\avidu\\Projects\\khelsutra\\web\\README.md",
      "C:\\Users\\avidu\\Projects\\khelsutra\\site\\server.js",
      "C:\\Users\\avidu\\Projects\\khelsutra\\site\\scripts\\droplet-smoke.sh"
    ]
  },
```

```
  "uiRepo": {
    "site_ui_surface": "khelsutra repo (cwd C:\\Users\\avidu\\Projects\\khelsutra, private, all-rights-reserved, master) has THREE UI surfaces, two live + one scaffold:\n\n1) site/ ? the PUBLIC marketing landing page. SHIPPED + LIVE on khelsutra.guru. Runtime-portable static-asset + Worker app: `site/src/index.js` (Cloudflare Worker `fetch`) and `site/server.js` (node:http twin) both call the same `handleRequest(request, store)` over a `store` adapter (`site/src/store.js`: `d1Store` | `sqliteStore`) [PORTABILITY.md:8-23]. Public pages: `site/public/index.html` (landing) + `site/public/capture.html` (recording checklist) + `site/public/og.png`. It hosts the segmented Request-Access lead form ? D1 (`site/schema.sql`, `access_requests`) [REQUEST_ACCESS_FORM.md:42-66]. README.md:13: \"site/ ? public marketing landing page (static; served at khelsutra.guru)\". Auto-deploys: merge to master ? Cloudflare Workers Builds (root site/) [NEXT_STEPS.md:9-12]. Also runs as a Node+SQLite twin live on the DO droplet at http://168.144.126.176:8787 (systemd khelsutra-site) ? the SAME droplet the engine lives on.\n\n2) web/ ? the ALPHA React SPA. SCAFFOLD-ONLY today: web/ contains just README.md (no code yet). README.md:14: \"web/ ? React SPA (alpha; Vite + React + TS + Tailwind; Cloudflare Pages build root)\"; web/README.md:1-2: \"SPA scaffold lands here at execution-plan milestone M1 (Vite + React + TS + Tailwind). Dev: talks to the engine at http://localhost:8000 via Vite proxy; no auth locally (DEV_USER_EMAIL).\" This is where the operator/per-rally UI belongs.\n\n3) control-plane/ ? (beta) Cloud Run FastAPI control plane; README.md:16 ? not yet created.\n\nHow the appliance \"operator console\" relates to the public marketing site: they are deliberately SEPARATE surfaces with a gating boundary baked into the planned topology. The public marketing page (site/, khelsutra.guru, Cloudflare Pages/Worker, $0) is UNAUTHENTICATED ? anyone can read it and submit the lead form. The operator console / appliance UI is a GATED APP: per HOSTING_PLAN.md:24-44 the alpha topology splits app.khelsutra.guru (Cloudflare Pages ? the web/ SPA, behind Cloudflare Access email-OTP allowlist) from api.khelsutra.guru (Cloudflare Tunnel ? FastAPI :8000 on the box). So the operator console is the web/ SPA sitting BEHIND Cloudflare Access, talking to the engine API ? NOT a public page. UI_PROPOSAL.md:33-37 confirms: SPA on static hosting, API on the box via tunnel, auth = Cloudflare Access email-allowlist (zero auth code; backend reads the authenticated-email header). Grounding caveat to record honestly: the engine's FastAPI today has NO auth (CORS '*'), so the Access/tunnel gate is the ONLY thing standing between the public internet and the appliance ? the console must assume the gate, not the app, is the tenancy boundary in alpha. The marketing site sells/funnels (Request-Access form captures GPU + storage answers = the BYO-compute/BYO-storage demand signal); the gated console operates the decoupled engine (the just-shipped storage + worker decoupling). A reasonable IA: keep the console as one or more authed screens in the SAME web/ SPA (Cloudflare Pages app root), distinct from site/ which stays the public Worker ? they already auto-deploy on separate roots (site/ Worker vs web/ Pages) so docs/web/ changes don't touch the live site/ Worker [PER_RALLY_SELECTION.md:108].",
    "per_rally_relation": "The just-merged docs/PER_RALLY_SELECTION.md (PR #11, status IN PROGRESS, owner-approved 2026-06-20) is the FIRST screen of the broader operator console ? it is the \"pick rallies ? reel\" workhorse, not the whole console. Concretely:\n\n- PER_RALLY_SELECTION.md is the RallyPicker: a hybrid selectable rally grid + deterministic query bar that rides EXISTING engine endpoints with ZERO new engine code for its MVP ? POST /api/query ? select segment_ids ? POST /api/compile ? GET /api/highlights [PER_RALLY_SELECTION.md:12, §4.1, D6]. It maps to alpha screen A5 Highlights (engine UI_PROPOSAL.md:47) and milestone M4 (UI_ALPHA_EXECUTION_PLAN.md), with its correction-loop extension realizing A4 Rally Review (UI_PROPOSAL.md:46).\n\n- An \"operator console\" for the DECOUPLED engine is a SUPERSET. PER_RALLY_SELECTION covers the curation-of-results step only; the console additionally needs the screens around it that UI_PROPOSAL.md:41-48 already enumerates ? A1 Matches (videos + verdict chips), A2 Upload & Process, A3 Job progress, A6 Report ? PLUS the genuinely new decoupled-era operations that PER_RALLY_SELECTION explicitly excludes: kicking off/monitoring worker infer/train jobs against a bucket, choosing a StorageBackend/bucket, per-video/per-run cost attribution, and weights/model versions. PER_RALLY_SELECTION is job-tethered by design (D10: carries result.video_id, no GET /api/videos) and deliberately punts multi-video home (P8, owner-gated), per-rally preview/timeline (P9), and the correction loop (P10) ? those punted milestones ARE the rest of the operator console.\n\n- So the relation is: per-rally selection = ONE screen (the reel-builder) of the console; the console wraps it with match list, upload, job-launch/monitor (the new decoupled worker verbs), report, and cost/model-version surfaces. The appliance tracked-doc should declare itself a peer-of PER_RALLY_SELECTION.md (and of REQUEST_ACCESS_FORM.md), reuse its verified engine-contract anchors (§3, §10) verbatim rather than re-deriving them, and adopt its honesty discipline (never surface motion-only server/receiver/events; never render a confidence meter from the Gemini string). It must NOT duplicate the RallyPicker design ? it should point to it as the embedded curation screen and own the job-orchestration/storage/cost surfaces PER_RALLY_SELECTION does not.",
    "doc_template": "Follow the engine repo's docs/PROJECT_DOC_TEMPLATE.md
```

(C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\PROJECT_DOC_TEMPLATE.md) ? the SAME template both existing khelsutra tracked docs mirror (PER_RALLY_SELECTION.md and REQUEST_ACCESS_FORM.md self-describe as \"mirrors the engine repo's PROJECT_DOC_TEMPLATE discipline\"). The exact structure to fill in:\n\nSTATUS HEADER (blockquote, 5 lines) ? > **Status:** `DRAFT` ? `<owner-gated? | in progress>` · **Owner:** Avi · **Created:** 2026-06-20 · **Last updated:** 2026-06-20 ; > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to docs/archives/ when DONE) ; > **Tracking anchors:** §7 progress tracker is the source of truth; indexed in docs/README.md; pointer in NEXT_STEPS.md + memory current-state once work starts ; > **Relation to existing docs:** `<peer-of PER_RALLY_SELECTION.md / REQUEST_ACCESS_FORM.md; extends UI_PROPOSAL.md alpha screens>` ; > **Honesty note:** mark each claim [verified] / [researched] / [design]; not legal/financial advice.\n\nBODY SECTIONS (delete OPTIONAL ones that don't apply):\n- §0 TL;DR (2?5 sentences: goal, approach, single most important caveat)\n- §1 Problem & goal (why now; concrete user outcome; links to read to get current)\n- §2 Decisions locked (table: # | Decision | Source / date | Implication) ? capture answered questions so they aren't relitigated\n- §3 Foundation ? research / prior art (OPTIONAL; include for engine-fact-heavy work, which this is ? use it for the verified engine/worker/storage contract, like PER_RALLY_SELECTION §3 did)\n- §4 Design / architecture (the plan + a REUSE MAP naming existing modules so new-vs-reused is explicit)\n- §5 Threat model / risk table (OPTIONAL ? relevant here: no engine auth, CORS '*', shared testbed droplet)\n- §6 Honest limits ? what this does NOT do (false-confidence guardrail)\n- §7 Deliverables & progress tracker ? SOURCE OF TRUTH (table: ID | Deliverable | Depends on | Gated? | Status | PR; legend ? Todo · ? In progress · ? Done · ? Blocked/gated; ONE small PR per row, branched from origin/master, NO stacking)\n- §8 Open questions ? owner / external (split blocking-gates from nice-to-knows; name who decides)\n- §9 Definition of done (explicit acceptance checklist; flip Status?DONE + archive when all ticked)\n- §10 References (Internal code anchors [verified] with paths; internal docs; external)\n- Changelog (dated entries at the bottom)\n\nHouse-voice rules from the template comment block (lines 9-16): §7 is the source of truth (one row per independently-shippable PR); keep it HONEST ? tag every claim [verified]/[researched]/[design], reflect real shipped state never aspirational; POINT to detail (code anchors), don't duplicate it; delete OPTIONAL sections that don't apply; move to docs/archives/ once Status=DONE (terminal-state only). After creating it, index it in khelsutra docs/README.md \"Tracked work (this repo)\" and add a pointer in khelsutra NEXT_STEPS.md (the same wiring PER_RALLY_SELECTION got).",

    "reuse_opportunities": [

      "Decoupled engine seams (just-shipped, master) ? the appliance's whole reason to exist: backend/storage/ StorageBackend ABC (base.py: fetch_to_local/put/exists/stat/list/delete + signed_url) with URI-driven local + s3 (AWS/DO Spaces/R2/MinIO) + gcs backends and lazy SDK import (__init__.py _backend_class), and the worker dispatcher backend/worker/__main__.py `python -m backend.worker {infer|train}` (pull from storage ? SAME pipeline building blocks ? push). The console drives THESE ? do not re-plumb storage or fork the pipeline; the worker is 'a thin orchestrator' over backend.main's blocks.",

      "RallyPicker design + its verified engine-contract anchors ? reuse docs/PER_RALLY_SELECTION.md §3/§4/§10 (POST /api/query ? segment_ids ? POST /api/compile ? GET /api/highlights; GET /api/config min_shot_count; GET /api/jobs/{id} result.video_id) verbatim instead of re-deriving; embed RallyPicker as the console's curation screen rather than redesigning it.",

      "Host-portability adapter pattern from PORTABILITY.md ? runtime-agnostic handleRequest(request, store) + d1Store/sqliteStore injection (site/src/store.js); the same 'logic-in-handler, host in adapter' shape is exactly how the StorageBackend registry already works in the engine, so the console can speak one URI contract across local/S3/GCS without host-specific branches.",

      "Parity + CI discipline ? site/scripts/parity-check.sh + .github/workflows/parity.yml (scheduled live byte-for-byte drift check between the Cloudflare host and the droplet twin) and test.yml (npm test green on every PR, blocks merge); reuse this dual-host-parity + green-CI-gates-merge pattern for any console screen that has both a live droplet and a Cloudflare path. node --test runs BOTH host paths (PORTABILITY.md:25-30).",

      "SPA scaffold + dev seam ? web/ (Vite + React + TS + Tailwind, Cloudflare Pages build root) with Vite proxy to engine :8000 and DEV_USER_EMAIL for no-auth local dev (web/README.md); the console screens land in this same scaffold (planned exec-plan M1) ? reuse the typed API client / useRallySelection() hook seams PER_RALLY_SELECTION P1-P2 introduce.",

      "Tracked-doc + workflow conventions ? docs/PROJECT_DOC_TEMPLATE.md structure, the docs/README.md index + NEXT_STEPS.md pointer wiring, REQUEST_ACCESS_FORM.md's storage/GPU questions as the demand signal that justifies the appliance, and the one-PR-per-row-off-origin/master no-stacking discipline.",

      "Gated-topology blueprint already written ? HOSTING_PLAN.md alpha topology (Cloudflare

```
Access email-OTP in front of app.khelsutra.guru Pages + api.khelsutra.guru Tunnel ? FastAPI on
the box, /api/health offline banner) is the ready-made gating model for the operator console;
reuse it rather than inventing auth, and note the engine-has-no-auth caveat as a risk."
    ],
    "file_refs": [
      "C:\\Users\\avidu\\Projects\\khelsutra\\README.md:11-16",
      "C:\\Users\\avidu\\Projects\\khelsutra\\web\\README.md:1-2",
      "C:\\Users\\avidu\\Projects\\khelsutra\\docs\\PER_RALLY_SELECTION.md:6,12,108,124",
      "C:\\Users\\avidu\\Projects\\khelsutra\\docs\\PORTABILITY.md:8-30",
      "C:\\Users\\avidu\\Projects\\khelsutra\\docs\\README.md:3-13",
      "C:\\Users\\avidu\\Projects\\khelsutra\\docs\\REQUEST_ACCESS_FORM.md:5-9,42-66,88-94",
      "C:\\Users\\avidu\\Projects\\khelsutra\\NEXT_STEPS.md:9-26,37-41",
      "C:\\Users\\avidu\\Projects\\khelsutra\\.github\\workflows\\parity.yml",
      "C:\\Users\\avidu\\Projects\\khelsutra\\.github\\workflows\\test.yml",
      "C:\\Users\\avidu\\Projects\\khelsutra\\site\\src\\store.js",
      "C:\\Users\\avidu\\Projects\\khelsutra\\site\\src\\index.js",
      "C:\\Users\\avidu\\Projects\\khelsutra\\site\\server.js",
      "C:\\Users\\avidu\\Projects\\khelsutra\\site\\scripts\\parity-check.sh",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\PROJECT_DOC_TEMPLATE.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\UI_PROPOSAL.md:41-60",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\HOSTING_PLAN.md:22-44",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\README.md:18-29",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\HANDOFF.md:5-30,50-64",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\storage\\base.py:14-49",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\storage\\__init__.py:28-45",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\worker\\__main__.py:1-16"
    ]
  }
}

Return a concrete, buildable design under this lens. mvp_scope = the smallest slice that drives
the full CUJ on a single box (CPU droplet acceptable via gemini/stub).
```

2. assistant (2026-06-20T09:03:02.685Z)

I'll design the KhelSutra single-box appliance. Let me first verify a few load-bearing facts in the codebase before committing to the design, since the research is detailed but I want to confirm the engine-side seams the design will lean on.

3. user (2026-06-20T09:03:04.855Z)

```
5c77f48 docs(decoupled): HANDOFF.md ? session checkpoint / resume-here (#255)
d5407eb feat(worker): Gen-0 train-from-bucket (worker train) on a CPU stub-trajectory mock
(#254)
814d813 feat(deploy): GPU machine provisioning ? NVMe scratch + native WASB setup (#253)
---BRANCH---
feat/native-wasb-runner
```

4. user (2026-06-20T09:03:04.986Z)

```
1       import os
2       import sys
3       import argparse
4       import logging
5       import uuid
6       import uvicorn
7       from dotenv import load_dotenv
8
9       load_dotenv()
```

```
10
11      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
12      logging.basicConfig(
13          level=logging.INFO,
14          format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
15          datefmt='%H:%M:%S'
16      )
17      logger = logging.getLogger(__name__)
18      from fastapi import FastAPI, HTTPException
19      from fastapi.middleware.cors import CORSMiddleware
20      from pydantic import BaseModel
21      from typing import List, Dict, Any, Optional
22
23      from fastapi.staticfiles import StaticFiles
24      from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
25      from backend.pipeline.validators.base import registry
26      from backend.infrastructure.database import Database
27      from backend.pipeline.segmenters.base import segmenter_registry
28      from backend.pipeline.stitcher.compiler import VideoCompiler
29      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
30      from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
31      from backend.config import load_config as load_app_config, AppConfig, StitchingConfig  #
new typed config
32      from backend.results import Failure  # standardized error/result contract
33      from backend.reporting import emit_indexer_report
34
35      app = FastAPI(title="Sports Highlight Indexer API")
36
37      # Enable CORS for the local web interface. allow_origins='*' with allow_credentials=True
is
38      # rejected by browsers per the fetch spec and is an unsafe default to carry into the
39      # auth/tenancy work; this tool uses no cookies, so credentials stay off.
40      app.add_middleware(
41          CORSMiddleware,
42          allow_origins=["*"],
43          allow_credentials=False,
44          allow_methods=["*"],
45          allow_headers=["*"],
46      )
47
48      # Serves static frontend files directly (now under api/ per approved structure)
49      os.makedirs("backend/api/static", exist_ok=True)
50      app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
51
52      @app.get("/", response_class=HTMLResponse)
53      def serve_index():
54          index_path = "backend/api/static/index.html"
55          if os.path.exists(index_path):
56              with open(index_path, "r", encoding="utf-8") as f:
57                  return f.read()
58          return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
59
60      # Global variables initialized at startup
```

5. assistant (2026-06-20T09:03:08.288Z)

Let me confirm the engine entrypoint, static-serving, and a couple of the key seams the design will reference.

6. user (2026-06-20T09:03:09.894Z)

```
620                     db=db_instance, video_id=video_id, video_path=args.video_path,
```

```
config=global_config,
621                     val_results=val_results, val_context=val_context,
622                     segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
623                     was_reingest=was_reingest, segmentation_ran=segmentation_ran,
624                 )
625                 if paths:
626                     print(f"[REPORT] Technical report : {paths.get('technical_md')}")
627                     print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
628                     print(f"[REPORT] Machine JSON     : {paths.get('json')}")
629             return
630
631         # Start Local Server Mode
632         if args.server or not args.video_path:
633             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
634             # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
635             # in-process wake-timer for any parked 'waiting' job died with the old process,
so
636             # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
637             _get_executor().submit(_drain_due_jobs)
638             uvicorn.run(app, host="127.0.0.1", port=args.port)
639
640     if __name__ == "__main__":
641         main()
642
```

7. user (2026-06-20T09:03:10.552Z)

```
1       """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3       The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4       "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5       It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6       Linux GPU box (GCP / a SaaS worker): it runs the **same** verified ``wasb_infer.py``
core
7       that the WSL adapter runs, so the trajectory CSV is byte-identical ? but WITHOUT the WSL
8       transport:
9
10        * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11          ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
12          (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13        * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps`` or ``cpu``; threaded
14          into ``wasb_infer.py --device`` (and so into the Hydra ``runner.device`` override).
15        * weights / repo / python / scratch come from the **environment** first
16          (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
17          ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
18
19      It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
20      name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
21      ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
22      the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
23      (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
24
25      **Acceptance gate (owner / GPU box):** byte-parity ? the SAME clip through the WSL
runner
26      (Windows) and this runner (Linux, ``device=cuda``) yields an identical trajectory CSV
and
27      identical candidate windows. That parity is a hardware check (a real GPU + the ``wasb``
28      env), so it is verified by the owner on the box ? not in this offline, subprocess-mocked
29      test suite.
30      """
31      import os
```

```
32      import shutil
33      import logging
34      import subprocess
35      from dataclasses import dataclass
36      from typing import Any, Dict, List, Optional, Tuple
37
38      from backend.pipeline.detectors.base import DetectorRunner
39      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
40      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
```

8. assistant (2026-06-20T09:03:14.936Z)

Interesting ? the `native_wasb_runner.py` already exists on this branch (`feat/native-wasb-runner`), which the research described as "the NEXT build." That's a useful nuance: M3.5 is in progress on this branch. This doesn't change the appliance design (the appliance must work CPU-only via stub/gemini regardless), but it confirms the native path is landing. I have enough verified grounding. Let me confirm the static UI entrypoint serving so the unified-origin design is accurate.

9. user (2026-06-20T09:03:17.079Z)

```
app.js
index.html
style.css
---DEPLOY---
README.md
bootstrap.sh
gpu_setup.sh
mount_scratch.sh
```

10. assistant (2026-06-20T09:03:26.345Z)

I have all the verified grounding I need. The key facts are confirmed: FastAPI binds `127.0.0.1:8000` (not 0.0.0.0 ? important for the exposure story), static SPA exists and is served from the engine, deploy scripts exist but no Caddy/nginx/Dockerfile yet, and the native WASB runner is actively being built on this branch. Now I'll compose the design.

11. user (2026-06-20T09:04:59.489Z)

```
<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be
parsed as JSON.
You sent (first 200 of 15557 bytes): {"name": "KhelSutra Appliance ? Reverse-Proxied Unified
Origin", "one_liner": "A reproducible single-box appliance where Caddy serves the operator-
console SPA and reverse-proxies /api/* to the no-auth
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters,
or truncated output. Retry with valid JSON.</tool_use_error>
```

12. assistant (2026-06-20T09:05:01.957Z)

The harness mangled my input into a wrapped `raw` string. I'll call StructuredOutput with the proper field structure directly.

13. user (2026-06-20T09:06:22.075Z)

```
<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be
parsed as JSON.
You sent (first 200 of 13845 bytes): {"name": "KhelSutra Appliance ? Reverse-Proxied Unified
Origin", "one_liner": "A reproducible single-box appliance where Caddy serves the operator-
console SPA and reverse-proxies /api/* to the no-auth
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters,
or truncated output. Retry with valid JSON.</tool_use_error>
```

14. assistant (2026-06-20T09:06:25.309Z)

The issue is my own input got serialized weirdly. I need to pass the actual JSON object as the tool arguments, not a string. Let me provide the fields as a proper object structure ? keeping values plain ASCII and avoiding any nested escaping.

15. user (2026-06-20T09:07:49.896Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 14773 bytes): {"name": "KhelSutra Appliance - Reverse-Proxied Unified Origin", "one_liner": "A reproducible single-box appliance where Caddy serves the operator-console SPA and reverse-proxies /api/* to the no-auth
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

16. assistant (2026-06-20T09:07:54.093Z)

I'm wrapping everything in a `raw` string key which is wrong. The tool expects the actual fields as native JSON object properties. Let me pass them directly as object fields without any `raw` wrapper.

17. user (2026-06-20T09:09:30.692Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 16557 bytes): {"name": "KhelSutra Appliance - Reverse-Proxied Unified Origin", "one_liner": "A reproducible single-box appliance where Caddy serves the operator-console SPA and reverse-proxies /api/* to the no-auth
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

18. assistant (2026-06-20T09:09:35.103Z)

I keep wrapping in `{"raw": ...}`. That's the bug. The schema wants top-level fields `name`, `one_liner`, `topology`, etc. directly. Let me write the JSON to a file first to validate it, then I know the exact bytes are clean, and pass the object correctly. First, let me construct it carefully as a real object.

19. user (2026-06-20T09:10:59.127Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 15182 bytes): {"name": "KhelSutra Appliance - Reverse-Proxied Unified Origin", "one_liner": "A reproducible single-box appliance where Caddy serves the operator-console SPA and reverse-proxies /api/* to the no-auth
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

20. assistant (2026-06-20T09:11:02.482Z)

I need to stop using the `raw` wrapper entirely. The tool input must be the object itself with the schema's named keys. Let me build the full JSON in a file to guarantee validity, then read it back to confirm, and finally pass the parsed structure. Let me write it out.

21. assistant (2026-06-20T09:12:32.948Z)

{
  "name": "KhelSutra Appliance - Reverse-Proxied Unified Origin",
  "one_liner": "A reproducible single-box appliance where Caddy serves the operator-console SPA and reverse-proxies /api/* to the no-auth engine FastAPI, with auth at the edge (Cloudflare Access or Caddy basicauth) so the engine never faces the public internet. Extends SETUP_NEW_MACHINE.md; drives the existing FastAPI loop today (gemini/stub on the CPU droplet) and the decoupled worker loop as native WASB lands, all from one origin.",
  "topology": "One box runs cooperating processes wired so ONLY the reverse proxy is public.\n\n(1) Caddy - the only process bound to public 0.0.0.0:443/80. Terminates TLS for

app.khelsutra.guru (automatic ACME), serves the built SPA as static files, and reverse-proxies /api/*, /static/*, /api/highlights/* to the engine. Auth lives HERE.\n\n(2) Engine FastAPI: `python -m backend.main --server`, which hardcodes uvicorn host=127.0.0.1:8000 (main.py:638, VERIFIED). That localhost bind is the load-bearing safety property: the no-auth API (CORS '*', zero Depends guard, main.py:42) is reachable ONLY via Caddy's proxy, never directly on the public NIC. systemd unit khelsutra-engine, Restart=always; GEMINI_API_KEY from /root/.keys (chmod 600), bucket creds from ~/.aws/credentials.\n\n(3) In-process job worker - already inside the engine: the max_workers=1 ThreadPoolExecutor + DB-jobs-table-as-clock (job_worker.py). No separate worker daemon for the local FastAPI loop. The decoupled `python -m backend.worker {infer|train}` CLI is on-demand (subprocess), not a long-lived service in MVP.\n\n(4) Storage = LocalStorage: storage.backend='local' is the default (config/models.py:298); fetch/put are identity/no-op passthroughs (local.py:33,42) so the box filesystem (output/uploads, output_dir) IS the store. No S3 needed on a single box.\n\nThe marketing site (/srv/khelsutra-site node, :8787) stays a SEPARATE systemd unit on a SEPARATE subdomain (khelsutra.guru), untouched. The appliance owns app.khelsutra.guru only.",

  "request_path": "ONE origin to the browser; two upstreams behind it. Browser -> https://app.khelsutra.guru (Caddy, 443, public). Caddy routing: (a) / and SPA assets -> served from disk (the built web/ bundle, file_server, SPA fallback to index.html); (b) /api/* -> reverse_proxy 127.0.0.1:8000 (engine); (c) /api/highlights/{video_id}/{filename} -> same engine proxy, which streams the compiled reel via FileResponse (main.py:307). Because everything is one origin, CORS is a non-issue for the SPA (same-origin fetch) and the engine's permissive CORS '*' is moot from the browser's side - the real boundary is Caddy. Source/proxy video for the REVIEW player is NOT served by the engine today (no streaming endpoint exists); MVP plays reels only (the compiled output via /api/highlights). Inline review of a 1080p proxy is a deliberate post-MVP add (a new GET /api/proxy/{video_id} streaming endpoint + Caddy passthrough), tracked, not faked. So: one origin, one TLS cert, one auth gate; the SPA and the API look like one site to the user even though they are two processes.",

  "compute_placement": "Same SPA + same /api contract, three honest compute backends - the difference is ONLY which segmenter/detector the box can run, surfaced from GET /api/config so the UI never offers a capability the box lacks.\n\n(A) CPU droplet (the live 168.144.126.176): NO GPU, NO local WASB. Honest options are (i) segmenter='gemini' (cloud brain, needs GEMINI_API_KEY, no GPU) for REAL rallies, or (ii) any hybrid with detector_impl=stub + skip_ai_handoff=true for a deterministic CPU MOCK (regression/demo only, NOT real detection). The UI shows a 'CPU box - real detection via Gemini; local WASB unavailable' badge from /api/config. This is the MVP target and it fully drives the CUJ.\n\n(B) GPU box (single Linux GPU droplet): segmenter='wasb_hybrid' with detector_impl='native' (the NativeWasbRunner now landing on branch feat/native-wasb-runner) runs real on-device shuttle detection, no WSL, no cloud. Same SPA, same /api/process; only /api/config advertises wasb_hybrid as available. The owner's Windows+WSL box is the same story via detector_impl='auto' (WSL).\n\n(C) Remote GPU worker via bucket: the decoupled path. The console box stays CPU; it flips storage.backend to s3 (DO Spaces), and a SEPARATE GPU box runs `python -m backend.worker infer --video-uri s3://... --out-uri s3://...` pulling/pushing the bucket. The console then reads outputs/<video_id>/ back from the bucket. This is OUT of MVP (needs new job-orchestration endpoints the engine does not have) but the design reserves the seam: the console's 'compute target' selector maps 1:1 to ComputeBackend (WHERE), which on a single box is the no-op 'local' choice (07 doc), so adding (C) later does not refactor the UI.\n\nThe ONE UI, three backends honesty rule: the console renders only what /api/config + a new GET /api/capabilities expose (available segmenters, whether a GPU/native runner is present, whether a Gemini key is set); it must never render a wasb_hybrid option on a CPU box or imply real detection when detector_impl=stub.",

  "auth_exposure": "Engine is no-auth BY DESIGN and stays that way; the gate is external. Two ratified-elsewhere options, pick per deployment:\n\n(1) Caddy edge auth (simplest, self-contained appliance): Caddy `basicauth` (bcrypt-hashed operator password in the Caddyfile) or `forward_auth` to an OIDC proxy. Single-tenant, invite-only, zero engine code. Good for the owner-operated alpha.\n\n(2) Cloudflare Access (the HOSTING_PLAN.md alpha model): cloudflared Tunnel dials OUT from the box to Cloudflare (NO inbound ports opened, home/box IP never exposed, HOSTING_PLAN.md:36); Cloudflare Access enforces an email-OTP allowlist in front of app.khelsutra.guru; the box trusts the Cf-Access-Authenticated-User-Email header / verifies the Access JWT. With Tunnel, Caddy can even bind 127.0.0.1 only and cloudflared is the sole ingress.\n\nThe NON-NEGOTIABLE invariant either way: the engine binds 127.0.0.1:8000 (already true, main.py:638) and the cloud firewall / DO firewall BLOCKS inbound 8000 (and 9000/9001/9023 MinIO/GCS-emulator test ports) so the raw API is unreachable from the internet. The gate is the proxy/tunnel, not the app. Go-live checklist (reuse HOSTING_PLAN.md:102-108): Access/basicauth ON, firewall closes 8000, narrow nothing in-app (CORS moot behind same-origin), rotate the previously-chat-shared GEMINI_API_KEY, set security.allowed_video_root to jail /api/process paths. Single-tenant token is the fallback if neither edge option is wanted (a static bearer the

SPA sends + a thin Caddy header check) - but edge auth is preferred because it keeps the engine truly auth-free.",
  "ui_console": "The operator console = authed screens in the EXISTING khelsutra web/ SPA (Vite+React+TS+Tailwind, Cloudflare Pages root), behind the edge gate, talking to /api on the same origin. It is a SUPERSET of PER_RALLY_SELECTION.md's RallyPicker (which it EMBEDS, not reduplicates). Screens:\n\n1. Upload & Process (A2) - file-picker that chunks <= max_part_mb via the EXISTING but un-wired /api/upload/init|part|complete (uploads.py), then feeds the returned absolute path into POST /api/process with a segmenter chosen from /api/capabilities. Wiring this is the single biggest net-new UI gap (app.js never calls upload today).\n\n2. Processing / Progress (A3) - polls GET /api/jobs/{id} every 3s, shows the free-text stage (hashing/validating/segmenting/finalizing) + a 'one job at a time' queue note (max_workers=1) + an honest 'failed: interrupted by server restart' state (fail_stale_jobs sweeps queued/running on boot).\n\n3. Rally index + selection - EMBED RallyPicker verbatim: POST /api/query -> select segment_ids -> POST /api/compile -> GET /api/highlights. Reuse PER_RALLY_SELECTION.md §3/§4/§10 anchors; honor min_shot_count from /api/config; NEVER surface motion-only server/receiver/events or a fake confidence meter.\n\n4. Reels / History - needs two SMALL net-new engine endpoints (GET /api/videos, GET /api/reels/{video_id}) so the console survives reload (today only get_job(id)/get_video(id) single-row lookups exist). Without them the library is amnesiac.\n\n5. Settings (compute + storage) - read-only in MVP: render /api/capabilities (segmenters available, GPU/native present, Gemini key set, storage backend). The storage/compute-target SELECTORS (local vs bucket vs remote worker) are the reserved decoupled-era seam, shown disabled with 'single-box: local' until the worker-orchestration endpoints exist.",
  "reuse_vs_new": "REUSES (do not re-plumb): the entire FastAPI loop /api/process -> /api/jobs -> /api/query -> /api/compile -> /api/highlights (built, main.py); the in-process max_workers=1 worker + DB-as-clock (job_worker.py); the chunked upload endpoints (uploads.py - built, just un-wired in app.js); StorageBackend ABC + LocalStorage default + the `python -m backend.worker` dispatcher (the decoupled seam for later remote GPU); the detector_impl seam (auto/native/stub) incl. the NativeWasbRunner now landing on this branch; the web/ SPA scaffold + RallyPicker design (PER_RALLY_SELECTION.md, embedded); the host-portability adapter pattern (handleRequest+store) and parity/test CI gates from the khelsutra repo; the gated topology from HOSTING_PLAN.md; SETUP_NEW_MACHINE.md / deploy/digitalocean/*.sh as the runbook base.\n\nNET-NEW (small, additive): a Caddyfile + a Caddy systemd unit + a khelsutra-engine systemd unit (NO Dockerfile exists anywhere today - README §96; either add Caddy+systemd OR a compose file, this design picks Caddy+systemd as the lighter reproducible path); GET /api/capabilities (derive available segmenters + GPU/native + key-present + storage backend from existing config/registry - thin); GET /api/videos and GET /api/reels/{video_id} (two small DB list queries - the only new engine data endpoints MVP needs); the web/ console screens (Upload wiring, Progress, Reels/History, Settings) around the embedded RallyPicker; a tracked project doc (peer of PER_RALLY_SELECTION.md, per PROJECT_DOC_TEMPLATE.md). Everything else is configuration, not code.",
  "mvp_scope": [
    "Caddyfile + two systemd units (khelsutra-engine, caddy) extending SETUP_NEW_MACHINE.md: one origin app.khelsutra.guru, TLS via ACME, /api/* + /static/* + /api/highlights/* -> 127.0.0.1:8000; engine stays bound to 127.0.0.1:8000 (already true); DO firewall blocks inbound 8000.",
    "Edge auth ON: Caddy basicauth (single operator) for the self-contained appliance, OR cloudflared Tunnel + Cloudflare Access email-allowlist per HOSTING_PLAN.md. Rotate the GEMINI_API_KEY before go-live.",
    "Wire browser upload in the web/ SPA: file-picker -> chunk <= max_part_mb via /api/upload/init|part|complete -> feed absolute path to POST /api/process (segmenter from /api/capabilities).",
    "GET /api/capabilities (thin): available segmenters, GPU/native-runner present?, Gemini key set?, storage backend - so the UI only offers what the box can do (CPU droplet => gemini real or stub mock; never wasb_hybrid).",
    "Drive the full CUJ on the CPU droplet end to end: upload -> POST /api/process with segmenter='gemini' (real rallies, needs key) -> poll /api/jobs -> RallyPicker (/api/query, select, /api/compile) -> download reel (/api/highlights). Stub+skip_ai_handoff is the no-key demo fallback.",
    "GET /api/videos + GET /api/reels/{video_id} (two small DB list endpoints) + a Reels/History screen so the console survives reload (today it is amnesiac on refresh).",
    "Embed RallyPicker (PER_RALLY_SELECTION.md) verbatim as the selection screen; reuse its verified engine-contract anchors; honor min_shot_count; no fake confidence/empty motion fields.",
    "A tracked project doc (PROJECT_DOC_TEMPLATE.md, peer of PER_RALLY_SELECTION.md) with §7 progress tracker, one small PR per row off origin/master, indexed in khelsutra docs/README.md +

```
NEXT_STEPS.md."
  ],
  "pros": [
    "Exposure-safe by construction: engine already binds 127.0.0.1 (main.py:638), so the no-auth
API is unreachable except via the proxy; auth at the edge means ZERO auth code in the engine and
no risk of a half-built in-app auth being the only guard.",
    "One origin = no CORS to design, one TLS cert, one URL for the operator; the engine's
permissive CORS '*' becomes irrelevant from the browser side.",
    "Maximally reuses shipped code: the whole /api loop, the in-process worker, the upload
endpoints, the storage+worker decoupling, the RallyPicker, and the SETUP_NEW_MACHINE runbook all
stand - MVP is mostly config + 3 thin endpoints + UI wiring.",
    "Honest about compute: the same SPA serves CPU droplet (gemini/stub), GPU box (native WASB),
and future remote-worker without UI forks, because it renders only /api/capabilities.",
    "Reproducible ops: Caddy+systemd extending the existing runbook is lighter than the
(nonexistent) Dockerfile path and matches how the box already runs the engine + marketing site
as systemd units.",
    "Does not fight the decoupled architecture: single box is the degenerate (storage=local,
compute=local) configuration of the same control/data/worker split; the storage-URI +
ComputeBackend seams are reserved, not contradicted."
  ],
  "cons": [
    "Two net-new engine endpoints (GET /api/videos, GET /api/reels/{video_id}) are genuinely
required for a non-amnesiac UI - the 'zero new engine code' claim that holds for RallyPicker
does NOT hold for the console as a whole.",
    "max_workers=1 ceiling: a 2nd concurrent operator/upload queues behind the 1st with no
queue-position surfaced; fine for single-tenant, visibly slow if shared.",
    "No Dockerfile means reproducibility leans on systemd+Caddyfile+bootstrap.sh being kept in
sync by hand (a runbook, not an image); drift risk across boxes.",
    "Inline review is reels-only in MVP - there is no proxy/source streaming endpoint, so the
operator cannot scrub the original before compiling; adding it is a real (tracked) follow-up,
not a config flip.",
    "Edge auth is the ENTIRE tenancy boundary: if Caddy/Access is misconfigured or the firewall
does not block 8000, the open engine is exposed - the go-live checklist must be enforced, not
assumed.",
    "Progress is coarse (free-text stage, poll-only, no %, no cancel) - the UX shows a stage
spinner, not a real progress bar, until SSE/cancel are added."
  ],
  "honest_caveats": [
    "The live droplet is CPU-only with NO GPU and NO local WASB. On that box, REAL rally
detection requires Gemini (cloud, needs a key, ships <=1280px chunks to Google's Files API -
must be in consent language); detector_impl=stub is a deterministic MOCK for demo/regression,
NOT real detection. Do not let the UI imply otherwise.",
    "The engine has ZERO app-layer auth and CORS '*' (main.py:42). Safety depends ENTIRELY on
the engine staying bound to 127.0.0.1 + a firewall blocking 8000 + the edge gate. If any of
those three slips, the box is wide open.",
    "No Dockerfile exists anywhere (verified). The 'appliance' is a runbook
(SETUP_NEW_MACHINE.md + Caddyfile + systemd units + bootstrap.sh), not a packaged image -
reproducibility is procedural.",
    "Uploaded originals under output/uploads are NEVER deleted today (VIDEO_LOCALITY_MODEL OQ1)
- a real privacy/disk hole on a publicly reachable box. MVP must add at least a retention sweep
or keep-proxy-only policy before exposing to non-owner users.",
    "GET /api/videos, GET /api/reels, GET /api/capabilities, browser-upload wiring, and a
retention sweep are NET-NEW and unbuilt - the console is not a pure UI-over-existing-API project
end to end.",
    "Native WASB (detector_impl=native) is landing on branch feat/native-wasb-runner but its
acceptance gate is byte-parity on a REAL GPU box, verified by the owner - it is NOT exercisable
on the CPU droplet, so the MVP single-box CUJ is proven only via gemini (real) or stub (mock).",
    "The remote-GPU-worker compute path (C) needs job-orchestration endpoints that DO NOT exist
(the FastAPI loop and the `backend.worker` bucket loop are separate today); the console's
storage/compute selectors are a reserved seam shown disabled, not a working MVP feature.",
    "Per-video Gemini/GPU cost has no ceiling or ledger today (no cost columns on the jobs
table); a single box can be surprise-billed - cap ai_max_workers=1 (already), serial uploads
(already), and add a per-video spend cap before wider exposure."
  ]
}
```

22. user (2026-06-20T09:12:32.964Z)

File created successfully at: C:\Users\avidu\Projects\badminton-highlight-indexer\artifacts\appliance_design.json (file state is current in your context ? no need to Read it back)

23. assistant (2026-06-20T09:12:39.240Z)

Now let me validate the JSON parses cleanly, then I'll pass the exact validated object to StructuredOutput.

24. user (2026-06-20T09:12:41.240Z)

OK keys: ['auth_exposure', 'compute_placement', 'cons', 'honest_caveats', 'mvp_scope', 'name', 'one_liner', 'pros', 'request_path', 'reuse_vs_new', 'topology', 'ui_console']
list lens: {'mvp_scope': 8, 'pros': 6, 'cons': 6, 'honest_caveats': 8}

25. assistant (2026-06-20T09:12:43.903Z)

The JSON is valid and has all 12 required keys. Now I'll pass it to StructuredOutput as the actual object (not wrapped in `raw`).

26. user (2026-06-20T09:13:49.334Z)

Structured output provided successfully

27. assistant (2026-06-20T09:14:02.002Z)

The structured design has been submitted successfully.

Summary of what was delivered: a buildable single-box appliance design under the reverse-proxied-unified-origin lens, grounded in verified engine facts (notably the localhost bind at `backend/main.py:638`, the no-auth/CORS-`*` posture at `main.py:42`, the local-storage default, the in-process `max_workers=1` worker, the un-wired upload endpoints, and the native WASB runner actively landing on the current branch `feat/native-wasb-runner`).

Key load-bearing decisions:
- **Topology/exposure:** Caddy is the only public process; the engine stays bound to `127.0.0.1:8000` with the DO firewall blocking 8000, so the no-auth API is reachable only via the proxy. Auth lives at the edge (Caddy basicauth or Cloudflare Access + Tunnel), never in the engine.
- **One UI, three honest compute backends:** the same SPA serves the CPU droplet (gemini real / stub mock), a GPU box (native WASB), and a future remote bucket worker ? gated by a thin new `GET /api/capabilities` so the UI never offers what the box can't do.
- **MVP CUJ on the CPU droplet:** upload ? process (`segmenter='gemini'`) ? poll ? embedded RallyPicker ? download reel, with `stub + skip_ai_handoff` as the no-key demo fallback.
- **Reuse vs new:** reuses the full `/api` loop, worker, upload endpoints, storage/worker decoupling, and RallyPicker; net-new is a Caddyfile + two systemd units, three thin endpoints (`/api/capabilities`, `/api/videos`, `/api/reels/{video_id}`), and the console screens.

Brutal-honesty caveats are recorded: CPU droplet means real detection needs Gemini (cloud, leaks ?1280px chunks); safety rests entirely on localhost-bind + firewall + edge gate; no Dockerfile exists (runbook, not image); uploads are never deleted (retention gap); and the remote-worker path needs orchestration endpoints that don't exist yet.

One scratch artifact was written to validate the JSON shape: `C:\Users\avidu\Projects\badminton-highlight-indexer\artifacts\appliance_design.json` (under the already-untracked `artifacts/` dir).